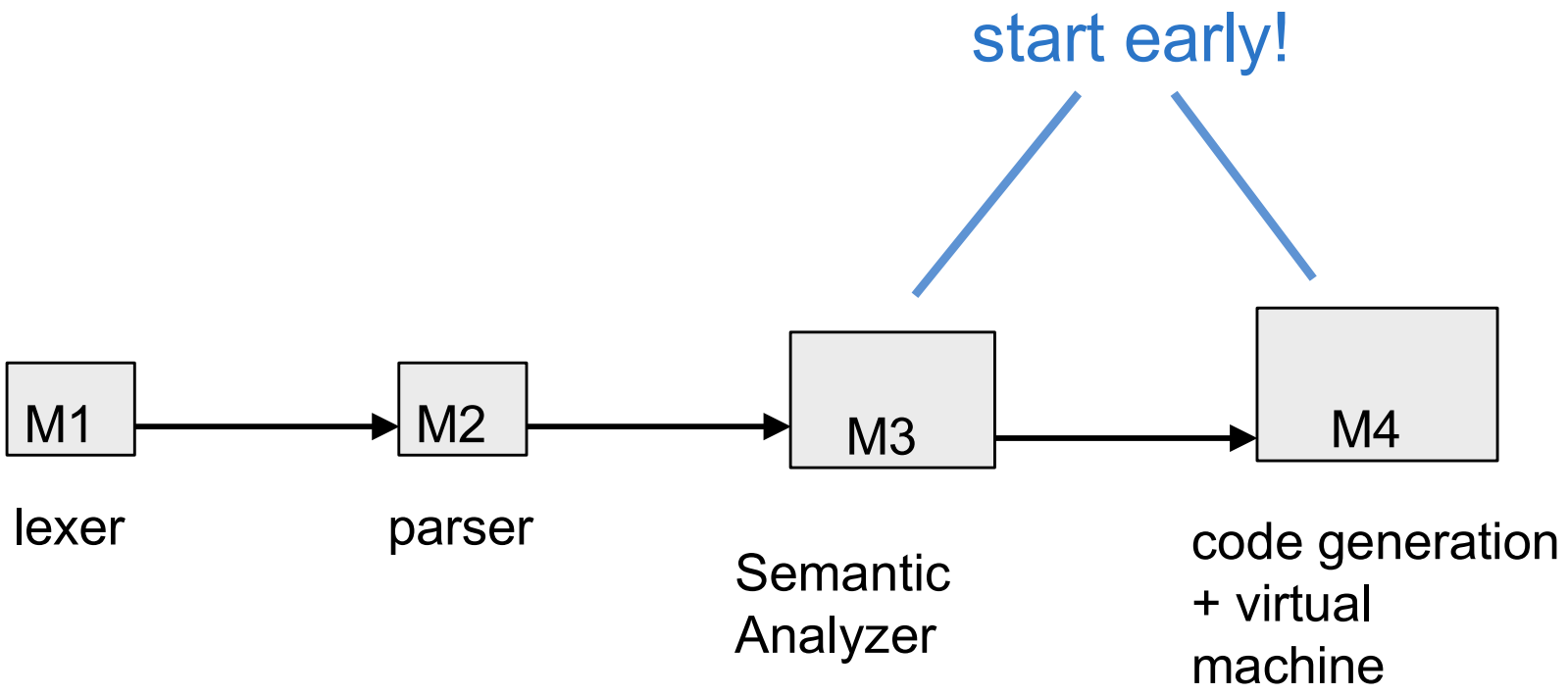
Compilers

The Goal of learning compiler

- Open the lid of compilers and see inside
  - Understand what they do
  - Understand how they work
  - Understand how to build them

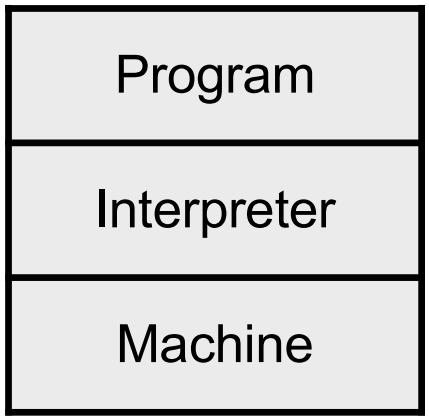
The Compiler Project

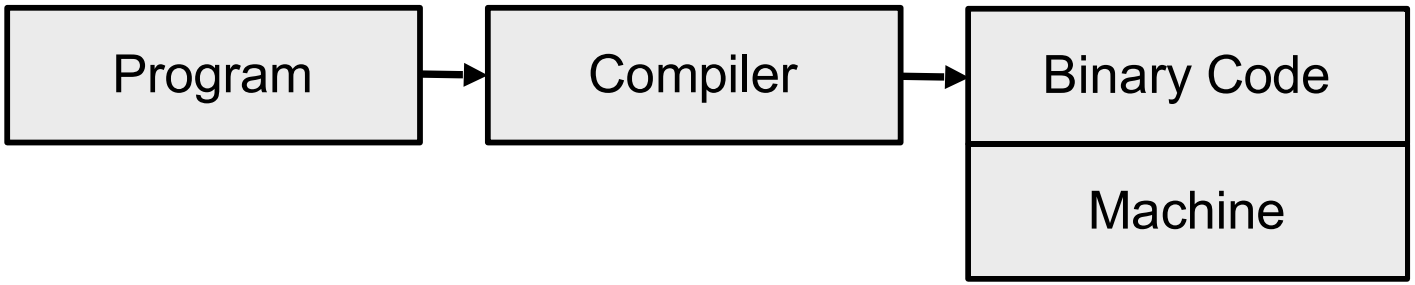
- You will write your own compiler!
- One big project
- … in 4 parts
- Start early



How are Languages Implemented?

- Two major strategies:
  - Interpreters run your program
  - Compilers translate your program





Language Implementations

- Compilers dominate low-level languages
  - C, C++, Go, Rust
- Interpreters dominate high-level languages
  - Python, JavaScript
- Many language implementations provide both
  - Java, Javascript, WebAssembly
  - Interpreter + Just in Time (JIT) compiler

History of High-Level Languages

- 1954: IBM develops the 704

- Problem
  - Software costs exceeded hardware costs!

- All programming done in assembly

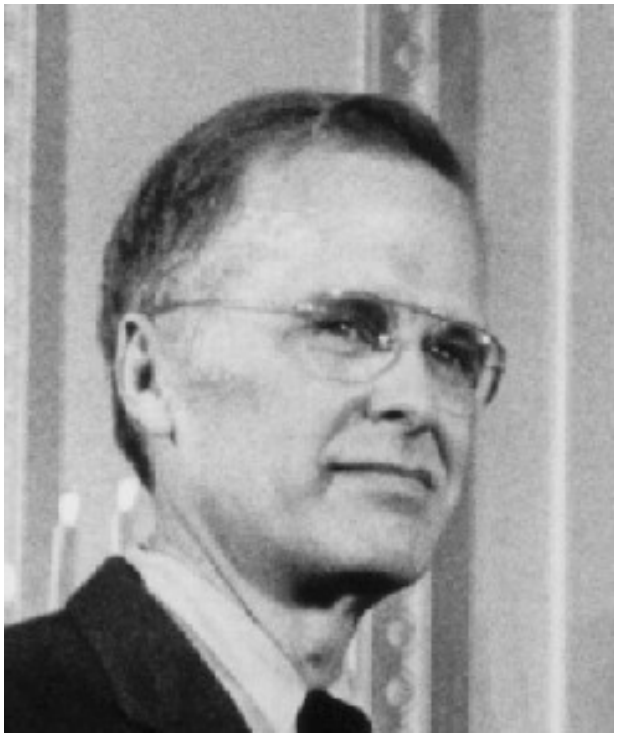
The Solution

- Enter “Speedcoding”
- An interpreter
- Ran 10-20 times slower than hand-written assembly

FORTRAN I

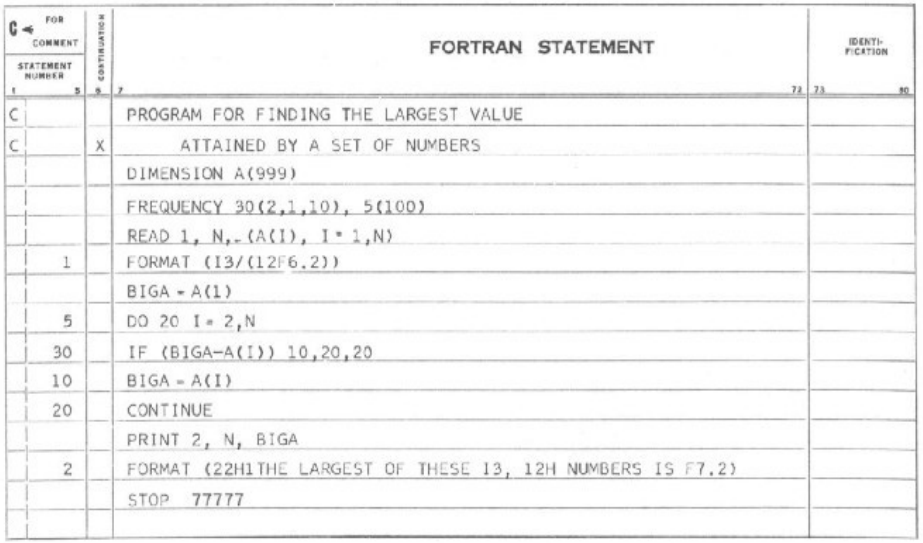
- Enter John Backus

- Idea
  - Translate high-level code to assembly
  - Many thought this impossible
  - Had already failed in other projects



FORTRAN I (Cont.)

- 1954-7
  - FORTRAN I project
- 1958
  - >50% of all software is in FORTRAN
- Development time halved
- Performance close to hand-written assembly!

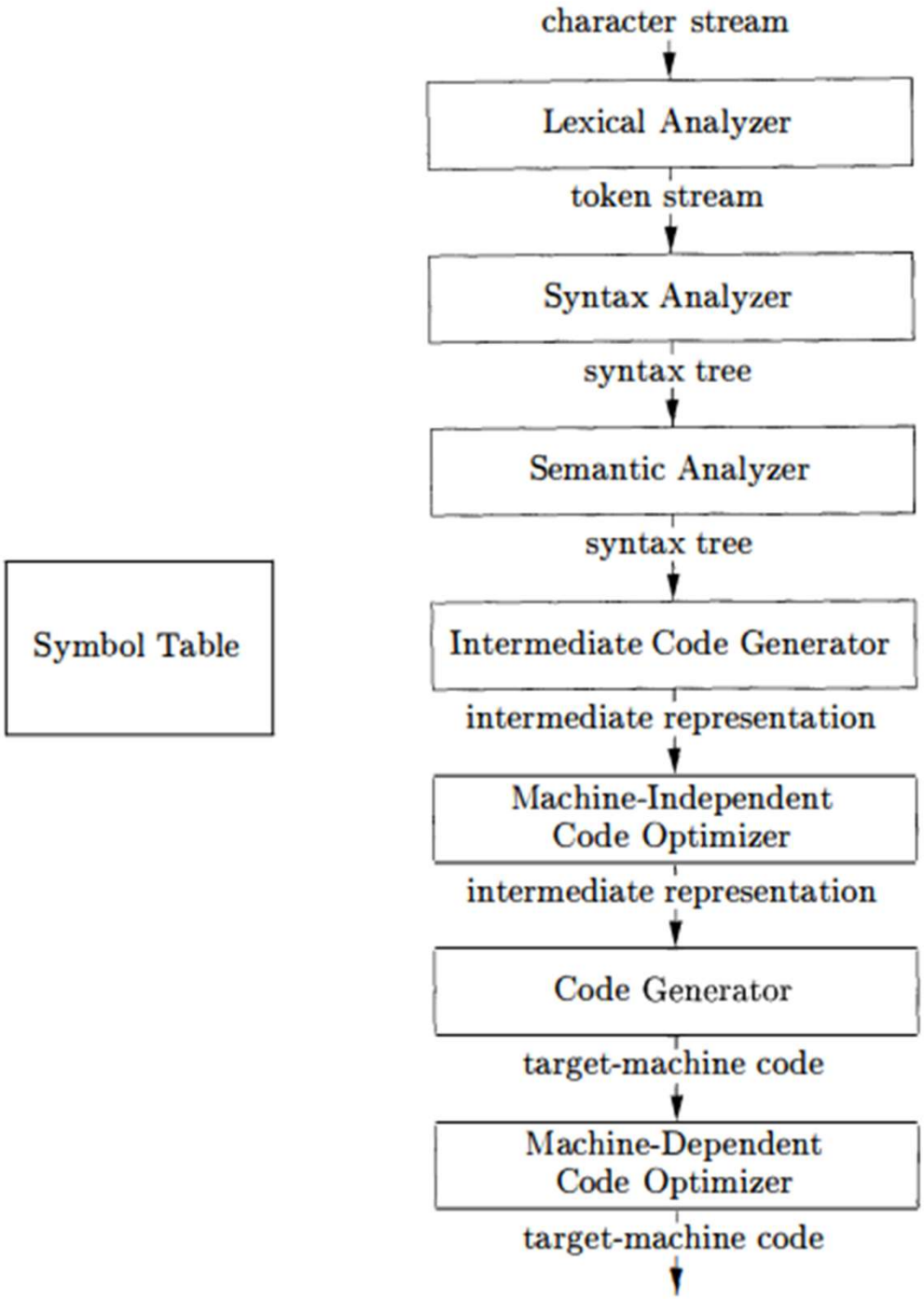


C ← FOR COMMENT	STATEMENT NUMBER (1–5)	CONTINUATION (6)	FORTRAN STATEMENT (7–72)	IDENTIFICATION (73–80)
C			PROGRAM FOR FINDING THE LARGEST VALUE	
C		X	ATTAINED BY A SET OF NUMBERS	
			DIMENSION A(999)	
			FREQUENCY 30(2,1,10), 5(100)	
			READ 1, N,(A(I), I=1,N)	
	1		FORMAT (I3/(12F6.2))	
			BIGA = A(1)	
	5		DO 20 I = 2,N	
	30		IF (BIGA-A(I)) 10,20,20	
	10		BIGA = A(I)	
	20		CONTINUE	
			PRINT 2, N, BIGA	
	2		FORMAT (22H1THE LARGEST OF THESE I3, 12H NUMBERS IS F7.2)	
			STOP 77777	

FORTRAN I

- The first compiler
  - Huge impact on computer science
- Led to an enormous body of theoretical and practical work
- Modern compilers preserve the outlines of FORTRAN I
- Can you name a modern compiler?

Compiler Achitecture



The Structure of a Compiler

1. Lexical Analysis — identify words
2. Parsing — identify sentences
3. Semantic Analysis — analyse sentences
4. Optimization — editing
5. Code Generation — translation

Can be understood by analogy to how humans comprehend English.

Lexical Analysis

- First step: recognize words.
  - Smallest unit above letters

This is a sentence.

More Lexical Analysis

- Lexical analysis is not trivial.

- Suppose we scramble the whitespaces:

  ist his ase nte nce

- Suppose we replace whitespace with z:

  iszthiszazsentence

And More Lexical Analysis

- Lexical analyzer divides program text into "words" or "tokens"

```
if x == y then z = 1; else z = 2;
```

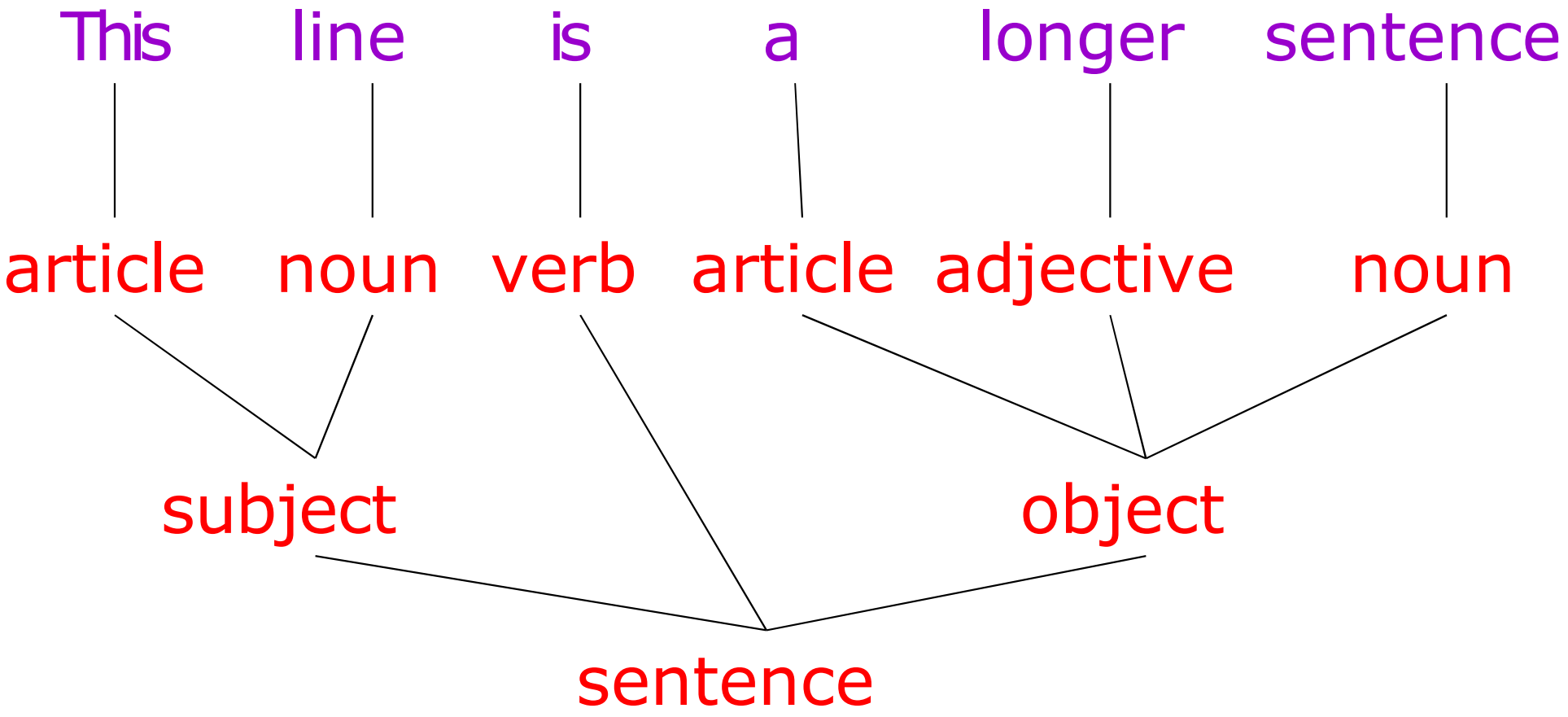
- Units:

Parsing

- Once words are understood, the next step is to understand sentence structure

- Parsing = Diagramming Sentences
  - The diagram is a tree

Diagramming a Sentence

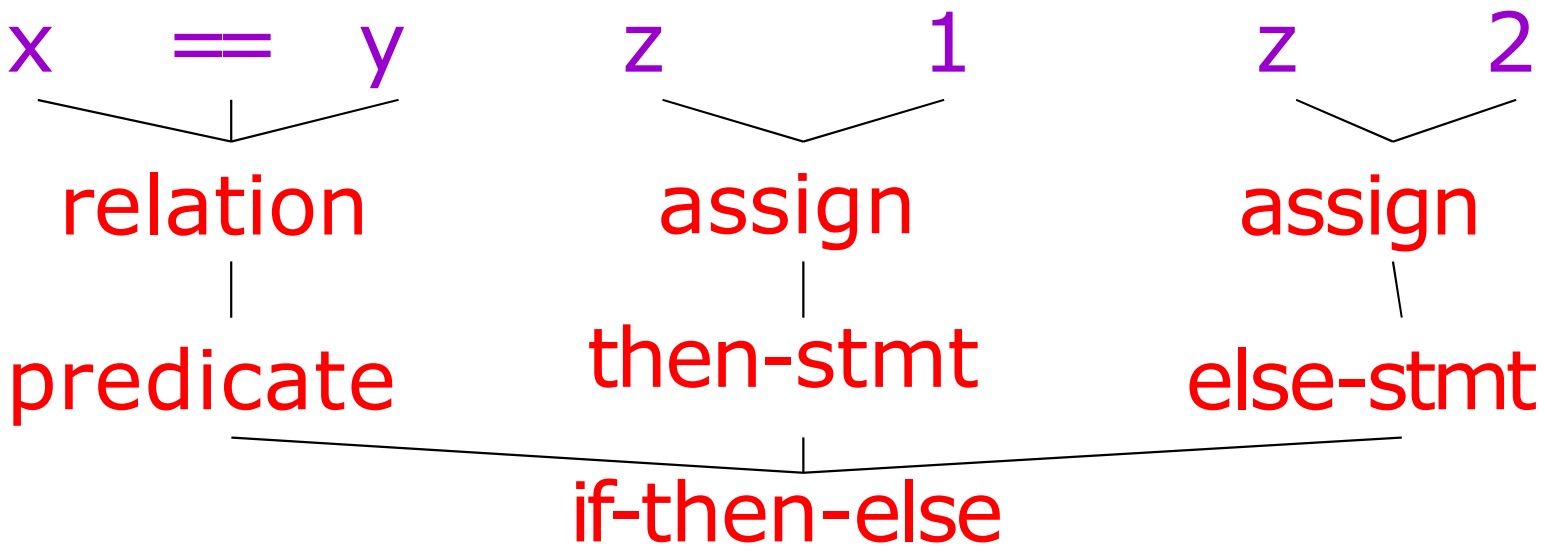


Parsing Programs

- Parsing program expressions is the same
- Consider:

$$\text{if } x == y \text{ then } z = 1 \text{ else } z = 2$$

- Diagrammed:



Semantic Analysis

- Once sentence structure is understood, we can try to understand “meaning”
  - But meaning is too hard for compilers

- Compilers perform limited semantic analysis to catch inconsistencies

Semantic Analysis in English

- Example:

  Jack said Jerry left his assignment at home.

  What does “his” refer to? Jack or Jerry?

- Even worse:

  Jill said Jill left her assignment at home?

  How many Jills are there?

  Which one left the assignment?

Semantic Analysis in Programming

- Programming languages define strict rules to avoid such ambiguities

- This C++ code prints "4"; the inner definition is used

```
{
   int i = 3;
   {
      int i = 4;
      cout << i;
   }
}
```

More Semantic Analysis

- Compilers perform many semantic checks besides variable bindings

- Example:

  Jack left her homework at home.

- Possible type mismatch between her and Jack
  - If Jack is male

Optimization

- Akin to editing
  - Minimize reading time
  - Minimize items the reader must keep in short-term memory

- Automatically modify programs so that they
  - Run faster
  - Use less memory
  - In general, to conserve some resource

Optimization Example

$x = y * 0$ is the same as $x = 0$

(the * operator is annihilated by zero)

Is this optimization legal?

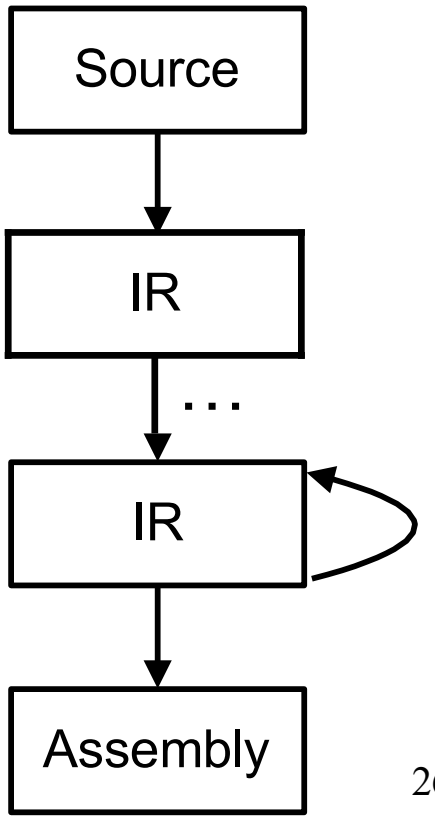
Code Generation

- Typically produces assembly code

- Generally a translation into another language
  - Analogous to human translation

Intermediate Representations (IR)

- Compilers typically perform translations between successive intermediate languages
  - All but first and last are intermediate representations (IR) internal to the compiler
- IRs are generally ordered in descending level of abstraction
  - Highest is source
  - Lowest is assembly



Intermediate Representations (IR) (Cont.)

- IRs are useful because lower levels expose features hidden by higher levels
  - registers
  - memory layout
  - raw pointers
  - etc.

- But lower levels obscure high-level meaning
  - Classes
  - Higher-order functions
  - Even loops…

Issues

- Compiling is almost this simple, but there are many pitfalls
- Example: How to handle erroneous programs?
- Language design has a big impact on the compiler
  - Determines what is easy and hard to compile
  - Course theme: many trade-offs in language design

Compilers Today

- The overall structure of almost every compiler adheres to our outline

- The proportions have changed since FORTRAN
  - Early: lexing and parsing most complex/expensive
  - Today: optimization dominates all other phases, lexing and parsing are well understood and cheap

- Compilers are now also found inside libraries:
  - XLA, TVM, Halide, DBMS, …

Parsing

PARSING

Two Methods :

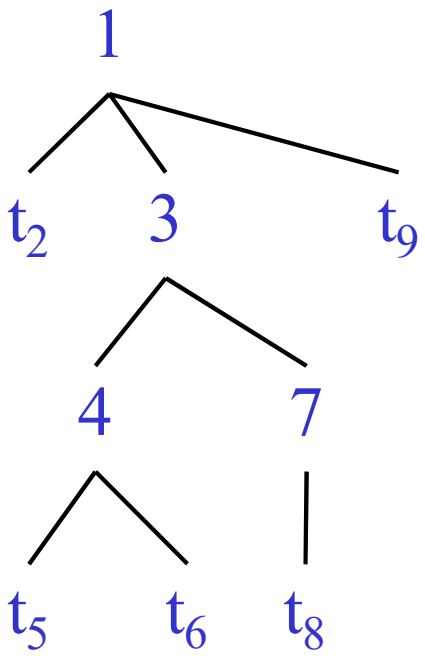
1. Top-Down Parsing
2. Bottom-up Parsing

Intro to Top-Down Parsing: The Idea

- The parse tree is constructed
  - From the top
  - From left to right

- Terminals are seen in order of appearance in the token stream:

t_2 t_5 t_6 t_8 t_9



Recursive Descent Parsing

- Consider the grammar

  $E \rightarrow T \mid T + E$

  $T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

- Token stream is: (int_5)

- Start with top-level non-terminal E
  - Try the rules for E in order

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

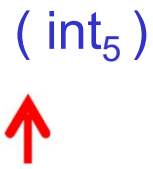
E

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$



Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

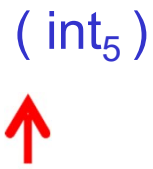
Mismatch: int is not (!
Backtrack …

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

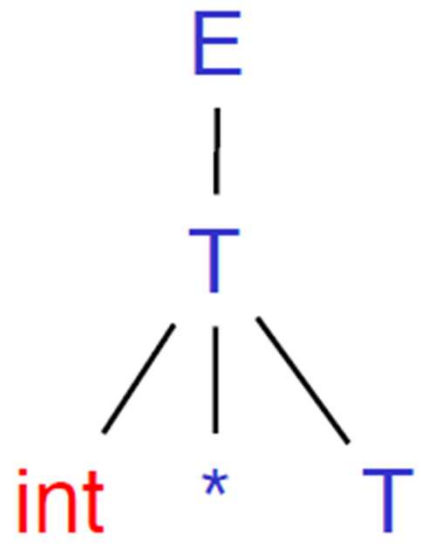
$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$



Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



Mismatch: int is not (!
Backtrack …

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

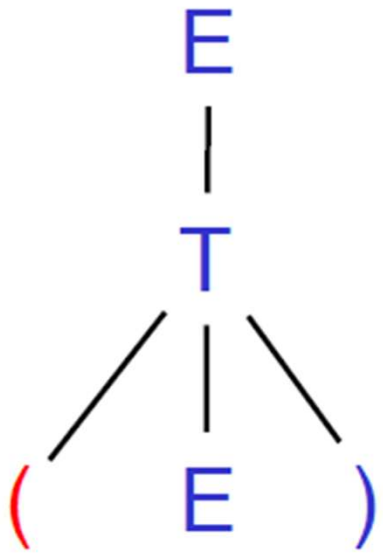
$T \rightarrow int \mid int * T \mid (E)$

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



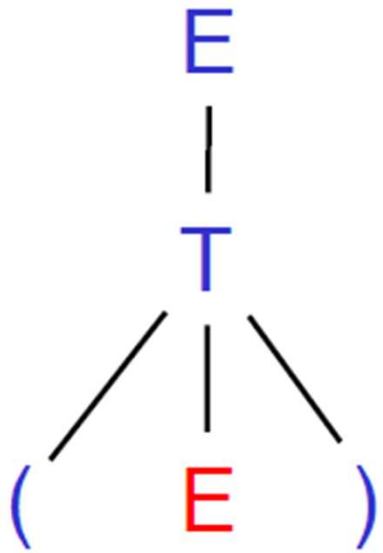
Match! Advance input.

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$

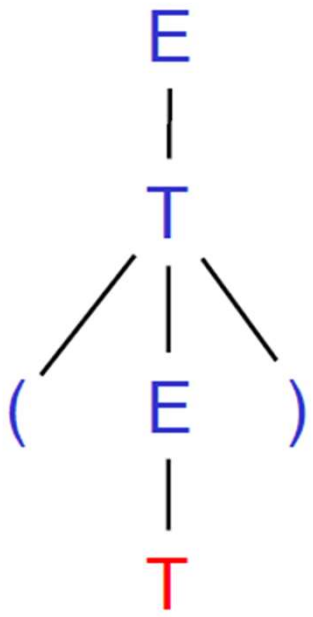


(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

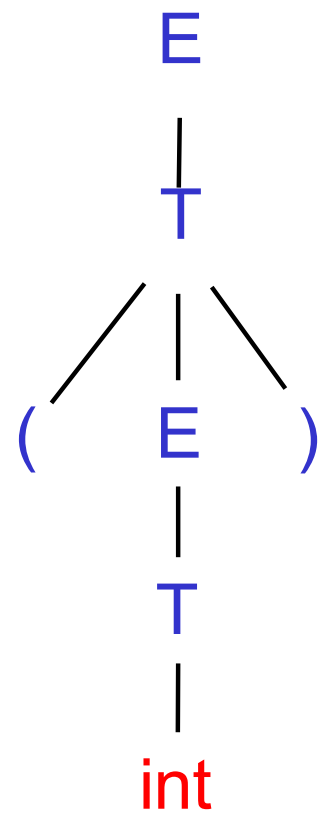


(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



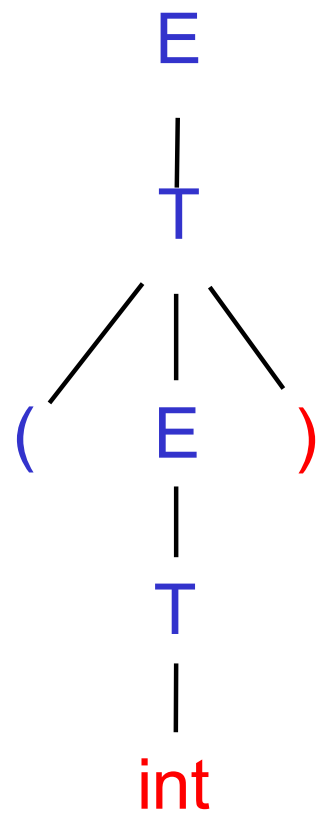
Match! Advance input.

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



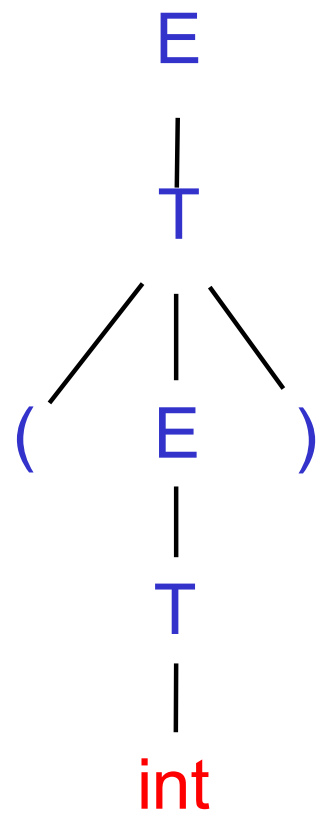
Match! Advance input.

(int_5)

Recursive Descent Parsing

$E \rightarrow T \mid T + E$

$T \rightarrow int \mid int * T \mid (E)$



End of input, accept.

(int_5)

A Recursive Descent Parser: Preliminaries

- Let TOKEN be the type of tokens
  - Special tokens INT, OPEN, CLOSE, PLUS, TIMES

- Let the global next point to the next token

A (Limited) Recursive Descent Parser (2)

- Define boolean functions that check the token string for a match of
  - A given token terminal

    bool term(TOKEN tok) { return *next++ == tok; }
  - The nth production of S:

    bool S_n() { … }
  - Try all productions of S:

    bool S() { … }

A (Limited) Recursive Descent Parser (3)

- For production $E \rightarrow T$

  bool E_1() { return T(); }

- For production $E \rightarrow T + E$

  bool E_2() { return T() && term(PLUS) && E(); }

- For all productions of E (with backtracking)

```
bool E() {
        TOKEN *save = next;
        return    (next = save, E1())
                || (next = save,  E2());   }
```

A (Limited) Recursive Descent Parser (4)

- Functions for non-terminal T

```
bool T1() { return term(INT); }
bool T2() { return term(INT) && term(TIMES) && T(); }
bool T3() { return term(OPEN) && E() && term(CLOSE); }

bool T() {
   TOKEN *save = next;
   return    (next = save, T1()
             || (next = save,  T2())
             || (next = save,  T3()); }
```

Recursive Descent Parsing. Notes.

- To start the parser
  - Initialize next to point to first token
  - Invoke E()

- Easy to implement by hand
  - But not completely general
  - Cannot backtrack once a production is successful
  - Works for grammars where at most one production can succeed for a non-terminal

Example

$E \rightarrow T \mid T + E$
$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$

(int)

```
bool term(TOKEN tok) { return *next++ == tok; }

bool E1() { return T(); }
bool E2() { return T() && term(PLUS) && E(); }

bool E()  {TOKEN *save = next; return     (next = save, E1())
                                       || (next = save,  E2());    }
bool T1() { return term(INT); }
bool T2() { return term(INT) && term(TIMES) && T(); }
bool T3() { return term(OPEN) && E() && term(CLOSE); }

bool T() { TOKEN *save = next;  return    (next = save, T1())
                                       || (next = save,  T2())
                                       || (next = save,  T3()); }
```

When Recursive Descent Does Not Work

- Consider a production $S \rightarrow S\ a$

  bool S_1() { return S() && term(a); }

  bool S() { return S_1(); }

- S() goes into an infinite loop

- A left-recursive grammar has a non-terminal S

  $S \rightarrow^+ S\alpha$ for some α

- Recursive descent does not work in such cases

Elimination of Left Recursion

- Consider the left-recursive grammar

$$S \rightarrow S\ \alpha \mid \beta$$

- S generates all strings starting with a β and followed by a number of α

- Can rewrite using right-recursion

$$S \rightarrow \beta\ S'$$

$$S' \rightarrow \alpha\ S' \mid \varepsilon$$

More Elimination of Left-Recursion

- In general

$$S \rightarrow S\ \alpha_1 \mid \ldots \mid S\ \alpha_n \mid \beta_1 \mid \ldots \mid \beta_m$$

- All strings derived from S start with one of $\beta_1,\ldots,\beta_m$ and continue with several instances of $\alpha_1,\ldots,\alpha_n$
- Rewrite as

$$S \rightarrow \beta_1\ S' \mid \ldots \mid \beta_m\ S'$$
$$S' \rightarrow \alpha_1\ S' \mid \ldots \mid \alpha_n\ S' \mid \varepsilon$$

General Left Recursion

- The grammar

  $S \rightarrow A\,\alpha \mid \delta$

  $A \rightarrow S\,\beta$

  is also left-recursive because

  $S \rightarrow^{+} S\,\beta\,\alpha$

- This left-recursion can also be eliminated

Summary of Recursive Descent

- Simple and general parsing strategy
  - Left-recursion must be eliminated first
  - … but that can be done automatically
- Historically unpopular because of backtracking
  - Was thought to be too inefficient
  - In practice, with some tweaks, fast and simple on modern machines
- Backtracking can be controlled by restricting the grammar